

Cryptography and Security Engineering

Complete Formula Handbook

Department of Computer Science and Engineering
CSE 4-1 Security Lab Reference

1 Modular Arithmetic & Number Theory Foundations

1.1 Modulo Operation & Canonical Remainder

For any integer $a \in \mathbb{Z}$ and positive modulus $m \in \mathbb{Z}^+$:

$$r = a \bmod m = a - \left\lfloor \frac{a}{m} \right\rfloor \cdot m, \quad 0 \leq r < m$$

If $a < 0$, the canonical non-negative remainder is:

$$r_{\text{pos}} = ((a \bmod m) + m) \bmod m$$

Basic modular properties:

$$(a + b) \bmod m = [(a \bmod m) + (b \bmod m)] \bmod m$$

$$(a \cdot b) \bmod m = [(a \bmod m) \cdot (b \bmod m)] \bmod m$$

1.2 Euclidean Algorithm for GCD

For non-negative integers a and b :

$$\gcd(a, b) = \begin{cases} a, & \text{if } b = 0 \\ \gcd(b, a \bmod b), & \text{if } b > 0 \end{cases}$$

Two integers a and b are coprime if and only if:

$$\gcd(a, b) = 1$$

1.3 Extended Euclidean Algorithm (Bézout's Identity)

For integers a and b , there exist integers x and y such that:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

If (x_1, y_1) is the recursive solution for $\gcd(b, a \bmod b)$, then:

$$x = y_1, \quad y = x_1 - \left\lfloor \frac{a}{b} \right\rfloor \cdot y_1$$

1.4 Modular Multiplicative Inverse

The modular multiplicative inverse $a^{-1} \pmod{m}$ satisfies:

$$(a \cdot a^{-1}) \equiv 1 \pmod{m}$$

The inverse exists if and only if $\gcd(a, m) = 1$. Using Extended Euclidean algorithm on (a, m) :

$$a \cdot x + m \cdot y = 1 \implies a^{-1} \equiv (x \bmod m + m) \bmod m$$

1.5 Fast Modular Exponentiation (Square-and-Multiply)

Calculates $b^e \bmod m$ in $\mathcal{O}(\log e)$ multiplications:

1. Initialize result = 1, base = $b \bmod m$.
2. While $e > 0$:
 - If $e \bmod 2 = 1 \implies \text{result} = (\text{result} \cdot \text{base}) \bmod m$.
 - $\text{base} = (\text{base}^2) \bmod m$.
 - $e = \lfloor e/2 \rfloor$.
3. Return result.

1.6 Euler's Totient Function & Euler's Theorem

- For prime p :

$$\phi(p) = p - 1$$

- For product of distinct primes $n = p \cdot q$:

$$\phi(n) = (p - 1)(q - 1)$$

Euler's Totient Theorem: If $\gcd(a, n) = 1$, then:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

2 Classical Ciphers

2.1 Caesar Cipher (Shift Cipher)

For alphabet space $\mathbb{Z}_{26} = \{0, 1, \dots, 25\}$ and key shift $k \in \mathbb{Z}_{26}$:

$$\text{Encryption: } C = (P + k) \bmod 26$$

$$\text{Decryption: } P = (C - k + 26) \bmod 26$$

2.2 Vernam Cipher (Stream Cipher / One-Time Pad)

- **Character Additive Stream:**

$$C_i = (P_i + K_i) \bmod 26$$

$$P_i = (C_i - K_i + 26) \bmod 26$$

- **Bitwise XOR Stream (One-Time Pad):**

$$C_i = P_i \oplus K_i$$

$$P_i = C_i \oplus K_i$$

2.3 Transposition Cipher

Plaintext characters are placed row-wise in matrix $M_{r \times c}$:

$$M_{i,j} = P_{i \cdot c + j}$$

Ciphertext is formed by reading columns according to permutation π :

$$C = \bigcup_{j=1}^c \text{Column}_{\pi(j)}$$

3 RSA Cryptosystem

3.1 Key Generation

1. Choose two large distinct prime numbers p and q .
2. Compute modulus:

$$n = p \cdot q$$

3. Compute Euler's totient:

$$\phi(n) = (p-1)(q-1)$$

4. Choose public exponent e such that:

$$1 < e < \phi(n) \quad \text{and} \quad \gcd(e, \phi(n)) = 1$$

5. Compute private exponent d :

$$d \equiv e^{-1} \pmod{\phi(n)} \iff (e \cdot d) \equiv 1 \pmod{\phi(n)}$$

Public Key: (e, n) **Private Key:** (d, n)

3.2 Encryption & Decryption

For plaintext message $m \in \mathbb{Z}_n$:

$$\text{Encryption: } c = m^e \bmod n$$

$$\text{Decryption: } m = c^d \bmod n$$

3.3 Mathematical Proof of Correctness

Since $e \cdot d \equiv 1 \pmod{\phi(n)} \implies e \cdot d = k \cdot \phi(n) + 1$:

$$c^d \equiv (m^e)^d \equiv m^{e \cdot d} \equiv m^{k \cdot \phi(n) + 1} \equiv (m^{\phi(n)})^k \cdot m \equiv 1^k \cdot m \equiv m \pmod{n}$$

3.4 RSA Digital Signature Scheme

For message m :

$$\text{Signing (Private Key } d): S = m^d \bmod n$$

$$\text{Verification (Public Key } e, n): m' = S^e \bmod n$$

The signature is **VALID** if and only if:

$$m' = m$$

3.5 RSA Multiplicative Homomorphism

Given ciphertexts $c_1 = m_1^e \bmod n$ and $c_2 = m_2^e \bmod n$:

$$c_{\text{comb}} = (c_1 \cdot c_2) \bmod n = (m_1^e \cdot m_2^e) \bmod n = (m_1 \cdot m_2)^e \bmod n$$

Decrypting the combined ciphertext directly recovers the product:

$$(c_{\text{comb}})^d \bmod n = (m_1 \cdot m_2) \bmod n$$

4 ElGamal Cryptosystem

4.1 Key Generation

1. Select prime p and generator (primitive root) $\alpha \in \mathbb{Z}_p^*$.
2. Choose private key $a \in [1, p - 2]$.
3. Compute public key parameter:

$$\beta = \alpha^a \bmod p$$

Public Key: (p, α, β) **Private Key:** a

4.2 Standard Encryption & Decryption

- **Encryption (Message $M \in \mathbb{Z}_p^*$):** Pick random ephemeral key $k \in [1, p-2]$ with $\gcd(k, p-1) = 1$:

$$c_1 = \alpha^k \bmod p$$

$$c_2 = (M \cdot \beta^k) \bmod p$$

$$\text{Ciphertext Pair} = (c_1, c_2)$$

- **Decryption:** Compute shared secret s :

$$s = c_1^a \bmod p = (\alpha^k)^a \bmod p = \beta^k \bmod p$$

Compute modular inverse $s^{-1} = (c_1^a)^{-1} \pmod{p}$.

Recover plaintext:

$$M = (c_2 \cdot s^{-1}) \bmod p = (c_2 \cdot (c_1^a)^{-1}) \bmod p$$

4.3 ElGamal Multiplicative Homomorphism

Given ciphertexts $C_1 = (\alpha^{k_1}, M_1 \beta^{k_1})$ and $C_2 = (\alpha^{k_2}, M_2 \beta^{k_2})$:

$$c_{\text{comb1}} = (c_{11} \cdot c_{21}) \bmod p = \alpha^{k_1+k_2} \bmod p$$

$$c_{\text{comb2}} = (c_{12} \cdot c_{22}) \bmod p = (M_1 \cdot M_2) \cdot \beta^{k_1+k_2} \bmod p$$

Decrypting $(c_{\text{comb1}}, c_{\text{comb2}})$ recovers:

$$M_{\text{result}} = (M_1 \cdot M_2) \bmod p$$

4.4 ElGamal Re-Randomization

Given ciphertext (c_1, c_2) and a fresh random scalar r :

$$c'_1 = (c_1 \cdot \alpha^r) \bmod p = \alpha^{k+r} \bmod p$$

$$c'_2 = (c_2 \cdot \beta^r) \bmod p = M \cdot \beta^{k+r} \bmod p$$

Decryption of (c'_1, c'_2) yields the exact same plaintext M .

4.5 Exponential ElGamal (Additive Homomorphism)

By embedding message M in exponent:

$$c_1 = \alpha^k \bmod p, \quad c_2 = (\alpha^M \cdot \beta^k) \bmod p$$

Multiplication of ciphertexts yields:

$$(c_{11} \cdot c_{21}, c_{12} \cdot c_{22}) \implies (\alpha^{k_1+k_2}, \alpha^{M_1+M_2} \cdot \beta^{k_1+k_2})$$

which corresponds to additive homomorphism over plaintexts.

4.6 ElGamal Digital Signature Scheme

1. **Signing** (Message M , Private key a , Random r with $\gcd(r, p-1) = 1$):

$$y_1 = \alpha^r \bmod p$$

$$y_2 = [r^{-1} \cdot (M - a \cdot y_1)] \bmod (p-1)$$

$$\text{Signature Pair} = (y_1, y_2)$$

2. **Verification** (Public Key (p, α, β) , Signature (y_1, y_2) , Message M): Verify whether:

$$\alpha^M \equiv (\beta^{y_1} \cdot y_1^{y_2}) \pmod{p}$$

Mathematical Proof:

$$\beta^{y_1} \cdot y_1^{y_2} \equiv (\alpha^a)^{y_1} \cdot (\alpha^r)^{r^{-1}(M - a \cdot y_1)} \equiv \alpha^{a \cdot y_1 + M - a \cdot y_1} \equiv \alpha^M \pmod{p}$$

5 Elliptic Curve Cryptography (ECC)

5.1 Weierstrass Elliptic Curve Equation

Over finite prime field $\mathbb{F}_p$ ($p > 3$):

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

Non-singularity discriminant condition:

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}$$

5.2 Identity Element & Point Inversion

- **Identity Element:** Point at infinity $\mathcal{O}$

$$P + \mathcal{O} = P$$

- **Point Inverse:** For point $P = (x_1, y_1)$:

$$-P = (x_1, (-y_1 \bmod p + p) \bmod p)$$

$$P + (-P) = \mathcal{O}$$

5.3 Point Addition & Point Doubling Formulas

Let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$. The resulting point is $R = P + Q = (x_3, y_3)$.

5.3.1 Slope (λ) Calculation

- **Point Addition** ($P \neq Q$):

$$\lambda = [(y_2 - y_1) \cdot (x_2 - x_1)^{-1}] \bmod p$$

- **Point Doubling** ($P = Q$):

$$\lambda = [(3x_1^2 + a) \cdot (2y_1)^{-1}] \bmod p$$

5.3.2 Resulting Coordinates (x_3, y_3)

$$\begin{aligned} x_3 &= (\lambda^2 - x_1 - x_2) \bmod p \\ y_3 &= [\lambda \cdot (x_1 - x_3) - y_1] \bmod p \end{aligned}$$

5.4 Scalar Multiplication (Double-and-Add Algorithm)

For positive integer k and point P :

$$kP = \underbrace{P + P + \dots + P}_{k \text{ times}}$$

Computed in $\mathcal{O}(\log k)$ point additions and doublings.

5.5 ECC ElGamal Public-Key Encryption

- **Domain Parameters:** Curve (a, b, p) , Base Point G , Order n .
- **Key Pair:** Private key $d \in [1, n - 1]$, Public key $Q = d \cdot G$.
- **Encryption (Plaintext mapped to Point P_m , Random $k \in [1, n - 1]$):**

$$C_1 = k \cdot G$$

$$C_2 = P_m + k \cdot Q$$

$$\text{Ciphertext Pair} = (C_1, C_2)$$

- **Decryption:**

$$P_m = C_2 - d \cdot C_1 = (P_m + k \cdot Q) - d(k \cdot G) = P_m + k(dG) - k(dG) = P_m$$

5.6 ECC Additive Homomorphic Property

For scalar messages m_1, m_2 embedded as points $P_1 = m_1G$ and $P_2 = m_2G$:

$$P_{\text{sum}} = P_1 + P_2 = m_1G + m_2G = (m_1 + m_2) \cdot G$$

5.7 ECDSA (Elliptic Curve Digital Signature Algorithm)

1. Signature Generation (Message M , Private key d , Base G , Order n):

(a) Choose random ephemeral integer $k \in [1, n - 1]$.

(b) Compute point $R = (x_R, y_R) = k \cdot G$.

(c) Compute r :

$$r = x_R \bmod n \quad (\text{if } r = 0, \text{ choose new } k)$$

(d) Compute s :

$$s = [k^{-1} \cdot (M + d \cdot r)] \bmod n \quad (\text{if } s = 0, \text{ choose new } k)$$

(e) Signature is the pair (r, s) .

2. Signature Verification (Public Key $Q = dG$, Signature (r, s) , Message M):

(a) Verify that $r, s \in [1, n - 1]$.

(b) Compute w :

$$w = s^{-1} \bmod n$$

(c) Compute u_1 and u_2 :

$$u_1 = (M \cdot w) \bmod n, \quad u_2 = (r \cdot w) \bmod n$$

(d) Compute point X :

$$X = (x_X, y_X) = u_1 \cdot G + u_2 \cdot Q$$

(e) Compute verification value v :

$$v = x_X \bmod n$$

(f) The signature is **VALID** if and only if:

$$v = r$$